C++23 STL Functions (C++17 and Later, GCC 14.2)

June 21, 2025

Introduction

This document summarizes the Standard Template Library (STL) functions available in C++17 and later (C++20, C++23), as supported by GCC 14.2 (MSYS2). Functions are organized by category, with concise notes on their purpose and version-specific enhancements. All functions are compatible with `-std=c++17`, `-std=c++20`, or `-std=c++23`.

1 Container-Specific Functions

STL container member functions for manipulation, access, and iteration.

- `at()`: Accesses element at index with bounds checking (throws `std::out_of_range`). E.g., `vec.at(2)`.
- `operator[]`: Accesses element at index without bounds checking (faster, unsafe). E.g., `map[key]`.
- `front()`: Returns first element (sequence containers like `std::vector`). Not for associative containers.
- `back()`: Returns last element (sequence containers). Not for associative containers.
- `data()`: Returns pointer to underlying array (`std::vector`, `std::string`, `std::array`).
- `begin()`, `end()`: Returns iterators to start and past-the-end of container.
- `cbegin()`, `cend()`: Returns const iterators for safe iteration.
- `rbegin()`, `rend()`: Returns reverse iterators for backward iteration.
- `crbegin()`, `crend()`: Returns const reverse iterators.
- `size()`: Returns number of elements in container.
- `max_size()`: Returns maximum possible elements (implementation-defined).
- `empty()`: Checks if container is empty (returns `true` if empty).
- `capacity()`: Returns allocated storage capacity (`std::vector`, `std::string`).
- `reserve(n)`: Requests capacity increase to at least `n` (`std::vector`, `std::string`).
- `shrink_to_fit()` (C++17): Requests capacity reduction to fit `size()` (non-binding).
- `insert()`: Adds element(s) at specified position or key. E.g., `vec.insert(pos, value)`.
- `erase()`: Removes element(s) at position or key. E.g., `set.erase(key)`.
- `clear()`: Removes all elements, leaving container empty.
- `emplace()`: Constructs element in place at specified position or key.
- `emplace_back()`: Constructs element at end of sequence container.
- `emplace_front()`: Constructs element at front (`std::list`, `std::deque`).
- `push_back()`: Adds element to end of sequence container.
- `pop_back()`: Removes last element of sequence container.

- `push_front()`: Adds element to front (`std::list`, `std::deque`).
- `pop_front()`: Removes first element (`std::list`, `std::deque`).
- `find()`: Searches for key in associative containers, returns iterator.
- `count()`: Counts elements matching a key (`std::multimap`, `std::multiset`).
- `lower_bound()`: Finds first element not less than key (associative containers).
- `upper_bound()`: Finds first element greater than key (associative containers).
- `equal_range()`: Returns range of elements matching key (associative containers).
- `contains()` (C++20): Checks if key exists in associative containers.
- `erase_if()` (C++20): Removes elements satisfying a predicate from any container.
- `erase()` (C++20): Removes elements equal to a value from any container.
- `extract()` (C++17): Extracts node from node-based containers (`std::map`, `std::set`).
- `merge()` (C++17): Merges elements from compatible container (`std::list`, `std::set`).
- `insert_or_assign()` (C++17): Inserts or updates key-value pair (`std::map`, `std::unordered_map`).
- `try_emplace()` (C++17): Conditionally constructs value if key doesnt exist (`std::map`).
- `insert_after()`, `erase_after()`, `splice_after()` (C++17): Manipulates elements after position (`std::forward_list`).
- `replace()` (C++23): Updates key-value pair in `std::flat_map`.
- `contains()` (C++23): Checks for substring or character in `std::string`, `std::string_view`.

2 Algorithms (`<algorithm>`)

General-purpose operations on iterator ranges or (C++20+) range objects.

2.1 Non-Modifying Sequence Operations

- `all_of`: Checks if all elements satisfy a predicate.
- `any_of`: Checks if any element satisfies a predicate.
- `none_of`: Checks if no elements satisfy a predicate.
- `for_each`: Applies a function to each element.
- `for_each_n` (C++17): Applies function to first `n` elements.
- `count`: Counts elements equal to a value.
- `count_if`: Counts elements satisfying a predicate.
- `mismatch`: Finds first position where two ranges differ.
- `find`: Finds first element equal to a value.
- `find_if`: Finds first element satisfying a predicate.
- `find_if_not` (C++17): Finds first element not satisfying a predicate.
- `find_first_of`: Finds first element matching any from another range.
- `find_end`: Finds last occurrence of a subsequence.
- `adjacent_find`: Finds first pair of equal adjacent elements.
- `search`: Finds first occurrence of a subsequence.
- `search_n`: Finds first sequence of `n` equal elements.

2.2 Modifying Sequence Operations

- `copy`: Copies elements from one range to another.
- `copy_if` (C++17): Copies elements satisfying a predicate.
- `copy_n`: Copies exactly `n` elements.
- `copy_backward`: Copies elements in reverse order.
- `move`: Moves elements to another range.
- `move_backward`: Moves elements in reverse order.
- `fill`: Fills range with a value.
- `fill_n`: Fills first `n` elements with a value.
- `transform`: Applies function to range, stores results.
- `generate`: Assigns generated values to range.
- `generate_n`: Assigns generated values to first `n` elements.
- `remove`: Moves non-matching elements to front (logical removal).
- `remove_if`: Moves elements not satisfying predicate to front.
- `remove_copy`: Copies non-matching elements to another range.
- `remove_copy_if`: Copies elements not satisfying predicate.
- `replace`: Replaces elements equal to a value.
- `replace_if`: Replaces elements satisfying a predicate.
- `replace_copy`: Copies range, replacing equal elements.
- `replace_copy_if`: Copies range, replacing predicate-satisfying elements.
- `swap`: Swaps two elements.
- `swap_ranges`: Swaps elements between two ranges.
- `iter_swap`: Swaps values pointed to by iterators.
- `reverse`: Reverses order of elements.
- `reverse_copy`: Copies range in reverse order.
- `rotate`: Rotates elements around a pivot.
- `rotate_copy`: Copies range with elements rotated.
- `shuffle`: Randomly reorders elements.
- `sample` (C++17): Randomly selects `n` elements.
- `unique`: Moves unique elements to front (logical duplicate removal).
- `unique_copy`: Copies unique elements to another range.
- `shift_left`, `shift_right` (C++20): Shifts elements left or right.

2.3 Sorting and Related Operations

- `sort`: Sorts range in ascending order (or custom comparator).
- `stable_sort`: Sorts preserving relative order of equal elements.
- `partial_sort`: Sorts first `n` elements.
- `partial_sort_copy`: Sorts and copies smallest `n` elements.
- `nth_element`: Partially sorts so `n`th element is in position.

- `is_sorted`: Checks if range is sorted.
- `is_sorted_until`: Finds first unsorted element.

2.4 Binary Search (Sorted Ranges)

- `lower_bound`: Finds first position for insertion of a value.
- `upper_bound`: Finds first position after a value.
- `equal_range`: Returns range of elements equal to a value.
- `binary_search`: Checks if value exists in sorted range.

2.5 Set Operations (Sorted Ranges)

- `merge`: Merges two sorted ranges into a new sorted range.
- `inplace_merge`: Merges two sorted subranges in place.
- `includes`: Checks if one sorted range contains another.
- `set_union`: Creates sorted union of two ranges.
- `set_intersection`: Creates sorted intersection of two ranges.
- `set_difference`: Creates sorted difference of two ranges.
- `set_symmetric_difference`: Creates sorted symmetric difference.

2.6 Heap Operations

- `make_heap`: Constructs a max-heap from a range.
- `push_heap`: Adds element to a heap.
- `pop_heap`: Moves largest element to end of heap.
- `sort_heap`: Converts heap to sorted range.
- `is_heap` (C++17): Checks if range is a heap.
- `is_heap_until` (C++17): Finds first element breaking heap property.

2.7 Min/Max Operations

- `min`: Returns smaller of two values.
- `max`: Returns larger of two values.
- `minmax` (C++17): Returns pair of smallest and largest values.
- `min_element`: Finds smallest element in range.
- `max_element`: Finds largest element in range.
- `minmax_element` (C++17): Finds smallest and largest elements.
- `clamp` (C++17): Restricts value to a specified range.

2.8 Permutation Operations

- `next_permutation`: Generates next lexicographical permutation.
- `prev_permutation`: Generates previous lexicographical permutation.
- `is_permutation` (C++17): Checks if one range is a permutation of another.

2.9 Range-Based Overloads (C++20)

All `<algorithm>` functions have `std::ranges::` equivalents, operating on range objects (e.g., `std::ranges::sort`, `std::ranges::find`).

3 Numeric Algorithms (<numeric>)

Mathematical operations on ranges.

- accumulate: Computes sum (or custom operation) of range with initial value.
- inner_product: Computes inner product of two ranges.
- adjacent_difference: Computes differences between adjacent elements.
- partial_sum: Computes partial sums of a range.
- iota: Fills range with increasing values.
- reduce (C++17): Parallelizable reduction (sum or custom).
- transform_reduce (C++17): Applies function and reduces result.
- inclusive_scan (C++17): Computes inclusive prefix sums.
- exclusive_scan (C++17): Computes exclusive prefix sums.
- transform_inclusive_scan (C++17): Transforms and computes inclusive sums.
- transform_exclusive_scan (C++17): Transforms and computes exclusive sums.
- ranges::accumulate, ranges::reduce, ranges::transform_reduce, ranges::inclusive_scan, ranges::exclusiv ranges::transform_inclusive_scan, ranges::transform_exclusive_scan (C++20): Range-based numeric operations.
- ranges::fold_left, ranges::fold_right, ranges::fold_left_with_iter, ranges::fold_left_first, ranges::fold_left_first_with_iter (C++23): Functional folding over ranges.

4 Iterators (<iterator>)

Utilities for iterator manipulation.

- advance: Moves iterator by specified distance.
- distance: Computes number of elements between iterators.
- next: Returns iterator advanced by distance.
- prev: Returns iterator moved backward by distance.
- iter_swap: Swaps values pointed to by two iterators.
- make_reverse_iterator: Creates reverse iterator from bidirectional iterator.
- make_move_iterator: Creates iterator for moving elements.
- front_inserter: Creates iterator for inserting at front (std::list).
- back_inserter: Creates iterator for inserting at back (std::vector).
- inserter: Creates iterator for inserting at specified position.
- ranges::begin, ranges::end, ranges::cbegin, ranges::cend, ranges::rbegin, ranges::rend, ranges::crbegin, ranges::crend (C++20): Range-based iterator access.

5 Function Objects (<functional>)

Functors for operations and comparisons.

- less: Checks if first argument is less than second.
- greater: Checks if first argument is greater than second.
- equal_to: Checks if arguments are equal.
- not_equal_to: Checks if arguments are not equal.

- `less_equal`: Checks if first argument is less than or equal to second.
- `greater_equal`: Checks if first argument is greater than or equal to second.
- `plus`: Adds two arguments.
- `minus`: Subtracts second argument from first.
- `multiplies`: Multiplies two arguments.
- `divides`: Divides first argument by second.
- `modulus`: Computes modulus of first by second.
- `negate`: Negates a single argument.
- `logical_and`: Computes logical AND of two arguments.
- `logical_or`: Computes logical OR of two arguments.
- `logical_not`: Computes logical NOT of an argument.
- `bit_and`: Computes bitwise AND of two arguments.
- `bit_or`: Computes bitwise OR of two arguments.
- `bit_xor`: Computes bitwise XOR of two arguments.
- `bit_not`: Computes bitwise NOT of an argument.
- `bind` (C++17): Binds arguments to a callable.
- `ref`, `cref` (C++17): Creates reference or const reference wrappers.
- `invoke` (C++17): Invokes callable with arguments.
- `bind_front`, `bind_back` (C++20): Binds arguments to front or back of callable.
- `invoke_r` (C++23): Invokes callable with specified return type.

6 Ranges (<ranges>, C++20 and Later)

Views and utilities for range-based operations.

- `views::all` (C++20): Adapts range into a view for pipeline operations.
- `views::filter` (C++20): Filters elements satisfying a predicate.
- `views::transform` (C++20): Applies function to each element.
- `views::take` (C++20): Takes first `n` elements.
- `views::drop` (C++20): Skips first `n` elements.
- `views::reverse` (C++20): Reverses element order.
- `views::elements` (C++20): Extracts tuple/pair elements by index.
- `views::keys` (C++20): Extracts keys from pair-like ranges.
- `views::values` (C++20): Extracts values from pair-like ranges.
- `views::zip` (C++23): Combines ranges into tuples element-wise.
- `views::zip_transform` (C++23): Combines ranges with transformation.
- `views::adjacent` (C++23): Produces tuples of adjacent elements.
- `views::adjacent_transform` (C++23): Applies function to adjacent elements.
- `views::chunk` (C++23): Splits range into chunks of specified size.
- `views::slide` (C++23): Produces sliding windows of specified size.
- `views::chunk_by` (C++23): Groups elements by predicate.

- `views::join_with` (C++23): Joins ranges with a delimiter.
- `views::cartesian_product` (C++23): Computes Cartesian product of ranges.
- `ranges::to` (C++23): Converts range to a container (e.g., `std::vector`).

7 Memory Management (`<memory>`)

Utilities for allocation and object lifetime.

- `make_unique` (C++17): Creates `std::unique_ptr` with new object.
- `make_shared` (C++17): Creates `std::shared_ptr` with new object.
- `allocate_shared` (C++17): Creates `std::shared_ptr` with custom allocator.
- `unique_ptr::reset` (C++17): Replaces managed object or sets to nullptr.
- `shared_ptr::use_count` (C++17): Returns number of shared owners.
- `addressof` (C++17): Gets true address, bypassing `operator&`.
- `uninitialized_copy` (C++17): Copies to uninitialized memory.
- `uninitialized_fill` (C++17): Fills uninitialized memory with value.
- `uninitialized_default_construct` (C++17): Default-constructs in uninitialized memory.
- `uninitialized_value_construct` (C++17): Value-constructs in uninitialized memory.
- `make_shared_for_overwrite`, `make_unique_for_overwrite` (C++20): Creates smart pointers with default-initialized objects.
- `to_address` (C++20): Gets raw pointer from smart pointer or iterator.
- `start_lifetime_as`, `start_lifetime_as_array` (C++23): Starts lifetime of object or array.
- `assume_aligned` (C++23): Hints at memory alignment for optimization.
- `to_underlying` (C++23): Converts enum to underlying type.

8 Utilities (`<utility>`, `<tuple>`)

General-purpose utilities.

- `swap`: Swaps values of two objects.
- `exchange`: Replaces value and returns old value.
- `forward`: Forwards argument preserving value category.
- `move`: Converts argument to rvalue for moving.
- `move_if_noexcept` (C++17): Conditionally moves if no exceptions.
- `as_const` (C++17): Converts to const reference.
- `make_pair` (C++17): Creates `std::pair` from two values.
- `make_tuple` (C++17): Creates `std::tuple` from values.
- `tie` (C++17): Creates tuple of lvalue references for unpacking.
- `forward_as_tuple` (C++17): Creates tuple of forwarded references.
- `get` (C++17): Accesses tuple or pair element.
- `tuple_element` (C++17): Gets type of tuple element.
- `tuple_size` (C++17): Gets number of tuple elements.
- `cmp_equal`, `cmp_not_equal`, `cmp_less`, `cmp_greater`, `cmp_less_equal`, `cmp_greater_equal` (C++20): Safe integral comparisons.

9 GCC 14.2 (MSYS2) Support

- **C++17**: All functions fully supported with `-std=c++17`.
- **C++20**: Range-based algorithms and views supported with `-std=c++20`.
- **C++23**: `std::flat_map`, `std::string::contains`, `std::ranges::fold_left`, and new views supported with `-std=c++23`; some features (e.g., `start_lifetime_as`) are experimental.
- **Verification**: Use `g++ -std=c++23 -E -x c++ /dev/null -dM | grep __cpp` to check feature support.